CESI
ÉCOLE D'INGÉNIEURS

Tests Unitaires PHPUnit

Tester TaskModel avec des Mocks et Stubs

Qu'est-ce qu'un test unitaire ?

CESI
ÉCOLE D'INGÉNIEURS

Un test unitaire est une procédure permettant de vérifier le bon fonctionnement d'une partie précise d'un logiciel ou d'une portion d'un programme (appelée « unité »).

Le principe en 2 étapes :

1

Exécuter du code

Appeler la méthode à tester avec des données contrôlées (entrées définies).

2

Vérifier le résultat

S'assurer que la valeur retournée correspond bien à la valeur attendue (assertion).

Test boîte blanche

Contrairement au test boîte noire (où on teste seulement les entrées/sorties), ici on connaît la structure interne du code (algorithmes, conditions, boucles, etc.) et on s'en sert pour concevoir les tests. On doit vérifier que **chaque partie du code fonctionne correctement**

Nombre de tests

Compter les return dans la méthode : 1 return = 1 test à écrire. Chaque chemin de code doit être couvert.

Un test unitaire = 1 méthode de test dans une classe qui étend TestCase. Son nom commence TOUJOURS par test (ex: testComputeTVA).

Pourquoi écrire des tests unitaires ?

CESI ÉCOLE D'INGÉNIEURS

Un test unitaire vérifie qu'une méthode d'une classe produit le résultat attendu, de façon ISOLÉE des autres composants.

Détecter les régressions

Après chaque modification du code, relancer les tests garantit qu'aucune fonctionnalité existante n'est cassée.

Documenter le code

Les tests décrivent le comportement attendu : ils servent de documentation vivante pour toute l'équipe.

Développement plus sûr

Détecter un bug immédiatement après écriture coûte 10x moins cher qu'en production.

Sécuriser le remplacement BDD

Modifier FileDatabase → MysqlDatabase sans casser TaskModel ? Les tests le garantissent.

Exemple complet — Tester la méthode computeTVA()

CESI ÉCOLE D'INGÉNIEURS

Classe Product (src/Entity/Product.php) avec méthode computeTVA() — 3 chemins → 3 tests.

src/Entity/Product.php

```php
<?php
namespace App\Entity;

class Product {
    const FOOD_PRODUCT = 'food';
    private $name, $type, $price;

    public function __construct($name, $type, $price) {
        $this->name  = $name;
        $this->type  = $type;
        $this->price = $price;
    }

    public function computeTVA() {
        if ($this->price < 0) {
            throw new \Exception('The TVA cannot be negative.');
        }
        if (self::FOOD_PRODUCT == $this->type) {
            return $this->price * 0.055; // TVA 5,5%
        }
        return $this->price * 0.196;     // TVA 19,6%
    }
}
```

tests/Entity/ProductTest.php

```php
<?php
namespace App\Tests\Entity;
use App\Entity\Product;
use PHPUnit\Framework\TestCase;
class ProductTest extends TestCase {
    // Test n°1 : produit alimentaire (type = food)
    public function testComputeTVAFoodProduct() {
        $product = new Product('Un produit',
            Product::FOOD_PRODUCT, 20);
        // 20 * 0.055 = 1.1
        $this->assertSame(1.1, $product->computeTVA());
    }
    // Test n°2 : autre type de produit
    public function testComputeTVAOtherProduct() {
        $product = new Product('Autre',
            'electronics', 20);
        // 20 * 0.196 = 3.92
        $this->assertSame(3.92, $product->computeTVA());
    }
    // Test n°3 : prix négatif → exception
    public function testNegativePriceComputeTVA() {
        $product = new Product('Prod',
            Product::FOOD_PRODUCT, -20);
        $this->expectException('Exception');
        $product->computeTVA();
    }
}
```

Règle : 1 return dans le code = 1 test. 1 throw = 1 test.

Ici **3 chemins → 3 tests**. assertSame vérifie valeur ET type (stricte).

Les Assertions PHPUnit

CESI
ÉCOLE D'INGÉNIEURS

Une assertion est une vérification : elle compare le résultat obtenu au résultat attendu. Si elle échoue → le test échoue (F).

Méthode	Description	Exemple
`assertSame(a, b)`	Vérifie valeur ET type (===)	`$this->assertSame(1.1, $result);`
`assertEquals(a, b)`	Vérifie la valeur (==, moins strict) (5 == "5"); // true → PHP convertit "5" en nombre	`$this->assertEquals('1.1', $result);`
`assertTrue(b)`	Vérifie que b est true	`$this->assertTrue($result);`
`assertFalse(b)`	Vérifie que b est false	`$this->assertFalse($result);`
`assertCount(n, tab)`	Vérifie la taille d'un tableau	`$this->assertCount(2, $tasks);`
`assertContains(v, t)`	Vérifie qu'un élément est dans le tableau	`$this->assertContains($task, $tasks);`
`assertNotContains(v,t)`	Vérifie l'absence dans le tableau	`$this->assertNotContains($done, $todos);`
`expectException(cls)`	S'attend à ce qu'une exception soit levée	`$this->expectException('Exception');`
`assertNull(val)`	Vérifie que la valeur est null	`$this->assertNull($user);`
`assertInstanceOf(cls,obj)`	Vérifie le type de l'objet	`$this->assertInstanceOf(Task::class, $obj);`

Tester les exceptions avec expectException()

CESI ÉCOLE D'INGÉNIEURS

Une méthode peut lever une exception au lieu de retourner une valeur. Ce cas doit aussi être testé. expectException() doit être appelée AVANT le code à tester.

src/Entity/Product.php — La méthode à tester

```php
<?php
// Méthode testée : computeTVA() lève une Exception si prix < 0
public function computeTVA() {
    if ($this->price < 0) {
        throw new \Exception('The TVA cannot be negative.');
    }
    // …
}
```

tests/Entity/ProductTest.php — testNegativePriceComputeTVA()

```php
<?php
public function testNegativePriceComputeTVA() {
    // ARRANGE : produit avec prix négatif
    $product = new Product(
        'Un produit',
        Product::FOOD_PRODUCT,
        -20     // prix négatif : déclenche l'exception
    );
    // ASSERT avant ACT (particularité des exceptions !)
    // On indique QUELLE exception on attend
    $this->expectException('Exception');
    // On peut aussi vérifier le message :
    // $this->expectExceptionMessage('The TVA cannot be negative.');
    // ACT : appeler la méthode — elle DOIT lever l'exception
    $product->computeTVA();
    // Si l'exception N'est PAS levée → test ÉCHOUE
    // Si l'exception EST levée → test PASSE
}
```

expectException('Exception')

Indique quelle classe d'exception est attendue. Utiliser le FQCN complet si dans un namespace (App\Exceptions\...).

*FQCN = Fully Qualified Class Name (nom de classe entièrement qualifié), on doit écrire **le chemin complet de la classe**, avec son namespace.

Exemple avec FQCN :

```
$this->expectException(\App\Exceptions\MyException::class);
```

Ordre IMPORTANT

expectException() doit TOUJOURS être appelée AVANT le code qui lève l'exception. Contrairement à assertSame(), ce n'est pas une vérification a posteriori.

Autres méthodes utiles

expectExceptionMessage('msg') : vérifie le message
expectExceptionCode(42) : vérifie le code d'erreur
expectExceptionMessageMatches('/regex/') : regex

Chaque throw dans le code = 1 test à écrire. Un throw est une sortie de méthode au même titre qu'un return.

Data Providers — Tester avec plusieurs jeux de données

CESI ÉCOLE D'INGÉNIEURS

Un data provider permet d'exécuter la même méthode de test plusieurs fois avec des données différentes, sans dupliquer le code.

tests/Entity/ProductTest.php — Data Provider

```php
<?php
class ProductTest extends TestCase {

    /**
     * @dataProvider pricesForFoodProduct
     * L'annotation @dataProvider lie ce test à la méthode fournissant les données.
     * La méthode sera appelée autant de fois qu'il y a de jeux de données.
     */
    public function testComputeTVAFoodProduct($price, $expectedTva) {
        $product = new Product('Un produit', Product::FOOD_PRODUCT, $price);
        $this->assertSame($expectedTva, $product->computeTVA());
    }

    // Le data provider : retourne un tableau de tableaux
    // Chaque sous-tableau = 1 appel de la méthode de test
    public function pricesForFoodProduct(): array {
        return [
            [0,   0.0],   // Appel 1 : $price=0,   $expectedTva=0.0
            [20,  1.1],   // Appel 2 : $price=20,  $expectedTva=1.1
            [100, 5.5],   // Appel 3 : $price=100, $expectedTva=5.5
        ];
    }
}

// Exécuter uniquement ce test (filtre par méthode) :
// vendor/bin/phpunit --filter=testComputeTVAFoodProduct
// → La méthode est exécutée 3 fois (une par jeu de données)
```

// cette méthode est le Data Provider

Pourquoi utiliser un data provider ?

Tester les cas limites sans dupliquer les méthodes de test.
Un seul endroit à modifier si la logique change.

Structure du provider

Retourne un tableau de tableaux.
Chaque sous-tableau = 1 set d'arguments.
L'ordre correspond aux paramètres de la méthode de test.

Comment PHPUnit sait appeler automatiquement le Data Provider

$product->computeTVA(); → On vérifie que la TVA calculée est correcte pour plusieurs prix.

@dataProvider pricesForFoodProduct → Utilise les données de pricesForFoodProduct pour exécuter ce test plusieurs fois.

testComputeTVAFoodProduct() →
À chaque exécution :

1. On crée un produit avec un prix,
2. On calcule la TVA,
3. On vérifie que le résultat = attendu

Cas limites à couvrir : 0, valeur négative, valeur maximale, chaîne vide, null. Les data providers sont parfaits pour ces scénarios.

Mock Objects — Créer et configurer avec createMock()

CESI ÉCOLE D'INGÉNIEURS

Un mock est un objet simulé qui remplace une dépendance réelle (BDD, API, service externe). Il permet de tester le code de façon isolée et rapide.

Isoler le code testé

Le mock simule la dépendance → les résultats du test ne dépendent que de la logique testée.

Contrôler le comportement

On définit exactement ce que retourne chaque méthode simulée pour chaque scénario.

Vérifier les interactions

Le mock enregistre les appels reçus : on peut vérifier qu'une méthode a bien été appelée.

Exemple basique — createMock() + willReturn()

```php
<?php
use PHPUnit\Framework\TestCase;

class MyTest extends TestCase {

    public function testMockExample() {
        // ── 1. Créer le mock ──────────────────────────────
        // createMock() génère une implémentation fictive de SomeClass
        // Toutes ses méthodes retournent null par défaut
        $mock = $this->createMock(SomeClass::class);

        // ── 2. Définir le comportement (stubbing) ─────────
        // Quand someMethod() est appelée, retourner 'mocked value'
        $mock->method('someMethod')
             ->willReturn('mocked value');

        // ── 3. Injecter le mock dans la classe testée ─────
        // On passe le mock à la place de la vraie dépendance
        $unitUnderTest = new ClassUnderTest($mock);

        // ── 4. Appeler la méthode à tester ────────────────
        $result = $unitUnderTest->performAction();

        // ── 5. Vérifier le résultat ───────────────────────
        $this->assertEquals('expected result', $result);
    }
}
```

Idée générale

On veut tester une classe (ClassUnderTest)
MAIS elle dépend d'une autre classe (SomeClass)

Problème

On ne veut pas utiliser la vraie classe (trop complexe, lente, ou dépend d'une base de données…)

Création du mock

$mock = $this->createMock(SomeClass::class); → PHPUnit crée un objet fake de SomeClass
Il ressemble à la vraie classe, mais il ne fait rien (toutes les méthodes retournent null)

$mock->method('someMethod') ->willReturn('mocked value');
→ On dit au mock : "Quand quelqu'un appelle someMethod(), retourne 'mocked value"

Sans mock : someMethod() → va faire un vrai calcul ou appel API
Avec mock : someMethod() → retourne juste "mocked value"
Donc, on contrôle totalement le comportement

Mock Objects — Créer et configurer avec createMock() …

CESI ÉCOLE D'INGÉNIEURS

Exemple basique – createMock() + willReturn()

```php
<?php
use PHPUnit\Framework\TestCase;

class MyTest extends TestCase {

    public function testMockExample() {
        // ── 1. Créer le mock ──────────────────────────────
        // createMock() génère une implémentation fictive de SomeClass
        // Toutes ses méthodes retournent null par défaut
        $mock = $this->createMock(SomeClass::class);

        // ── 2. Définir le comportement (stubbing) ─────────
        // Quand someMethod() est appelée, retourner 'mocked value'
        $mock->method('someMethod')
             ->willReturn('mocked value');

        // ── 3. Injecter le mock dans la classe testée ─────
        // On passe le mock à la place de la vraie dépendance
        $unitUnderTest = new ClassUnderTest($mock);

        // ── 4. Appeler la méthode à tester ────────────────
        $result = $unitUnderTest->performAction();

        // ── 5. Vérifier le résultat ───────────────────────
        $this->assertEquals('expected result', $result);
    }
}
```

createMock(Class::class)

Crée un faux objet. Toutes les méthodes retournent null par défaut.
Ne vérifie PAS les appels.

->method('nom')->willReturn(val)

Définit ce que retourne la méthode simulée. Peut aussi : willThrowException(), willReturnArgument()…

createStub() vs createMock()

createStub() → simuler un comportement (retour de valeur)
createMock() → simuler + vérifier le comportement (les appels)

createMock() est utilisé quand on veut AUSSI vérifier les appels (expects).
createStub() suffit quand on veut juste contrôler les valeurs retournées.

createStub

$service = $this->**createStub**(Service::class);
$service->method('getValue') ->willReturn(10);

Peu importe si getValue() est appelée 0 fois ou 100 fois, juste contrôler le **résultat**

createMock

$service = $this->**createMock**(Service::class);
$service->expects($this->once())
  ->method('getValue')
  ->willReturn(10);
Si getValue() n'est pas appelée → test échoue
Si elle est appelée 2 fois → test échoue

Mock Objects — Vérifier les interactions avec expects()

CESI ÉCOLE D'INGÉNIEURS

expects() vérifie QUE et COMMENT une méthode a été appelée. C'est la différence fondamentale entre un mock (expects) et un stub (willReturn seul).

Exemple PaymentProcessor — expects()->once()

```php
<?php
// — Exemple : PaymentProcessor dépend de PaymentGateway —
class PaymentProcessor {
    public function __construct(private PaymentGateway $gateway) {}
    public function processPayment(float $amount): bool {
        return $this->gateway->charge($amount);
    }
}

// — Test : charge() doit être appelée exactement 1 fois —
public function testProcessPaymentCallsCharge() {
    $gatewayMock = $this->createMock(PaymentGateway::class);

    // expects() : vérifier le nombre d'appels
    $gatewayMock->expects($this->once())    // ← appelée exactement 1x
        ->method('charge')
        ->with(100.0)                        // ← avec cet argument précis
        ->willReturn(true);                  // ← retourne true

    $paymentProcessor = new PaymentProcessor($gatewayMock);
    $paymentProcessor->processPayment(100.0);
    // PHPUnit vérifie automatiquement expects()->once() à la fin
    // → Si charge() n'est PAS appelée → test ÉCHOUE
    // → Si charge() est appelée 2x → test ÉCHOUE
    // → Si appelée avec 50.0 au lieu de 100.0 → test ÉCHOUE
}
```

`$this->once()`	`$this->exactly(n)`
Appelée exactement 1 fois	Appelée exactement n fois
`$this->never()`	**`$this->atLeastOnce()`**
Jamais appelée (0 fois)	Au moins 1 fois
`->with(val)`	**`->equalTo('str')`**
Appelée avec cet argument	Argument égal à la valeur

Bonne pratique : ne vérifier les interactions (expects) que quand c'est réellement important. Trop de expects rend les tests fragiles et difficiles à maintenir.

Exemple réel — Mock d'un Repository (UserService)

CESI ÉCOLE D'INGÉNIEURS

UserService dépend de UserRepository. Pour tester getUserName() en isolation, on mocke le repository — aucune BDD nécessaire.

src/Service/UserService.php

```php
<?php
// ── La classe à tester : UserService ──────────────────
class UserService {
    public function __construct(
        private UserRepository $repository
    ) {}

    public function getUserName($id): string {
        $user = $this->repository->find($id);
        return $user->name;
    }
}
```

Ce pattern s'applique directement à MyTodoo : TaskModel mocke FileDatabase (ou MysqlDatabase) → les tests ne touchent jamais la base de données réelle.

tests/Service/UserServiceTest.php

```php
<?php
// ── Le test : mocker UserRepository ──────────────────
class UserServiceTest extends TestCase {

    public function testGetUserName() {
        // Créer un mock de UserRepository
        $mockRepo = $this->createMock(
            UserRepository::class
        );

        // Configurer : find() retourne un faux utilisateur
        $mockRepo->method('find')
            ->willReturn((object) ['name' => 'John Doe']);
```

→

```php
// Injecter le mock dans UserService
        $service = new UserService($mockRepo);

        // Vérifier que getUserName retourne 'John Doe'
        $this->assertEquals(
            'John Doe',
            $service->getUserName(1)
        );
        // Aucune vraie BDD n'a été touchée !
    }
}
```

Pattern : Tester un Service avec un Repository mocké

1. Identifier la dépendance (UserRepository)
2. Créer le mock avec createMock(UserRepository::class)
3. Configurer : method('find')->willReturn(faux objet)
4. Injecter le mock dans UserService
5. Appeler la méthode testée : getUserName(1)
6. Assertion : assertEquals('John Doe', $result)

Avantages :

→ Test rapide : pas de requête SQL réelle

→ Test isolé : ne dépend pas de la BDD

→ Test fiable : même résultat à chaque exécution

→ Test maintenable : facile à lire et comprendre

Bonnes pratiques et récapitulatif des tests unitaires

CESI ÉCOLE D'INGÉNIEURS

Bonnes pratiques

- Nommer les méthodes clairement : testGetUserNameReturnsJohnDoe
- 1 méthode de test = 1 comportement précis à vérifier
- Couvrir tous les chemins (return + throw) sans viser 100%
- Mocke uniquement ce qui est nécessaire — pas tout
- Tester le comportement, pas l'implémentation interne
- Lancer les tests AVANT et APRÈS chaque modification

Récapitulatif des concepts

`Test unitaire`	Vérifie 1 méthode, 1 comportement, en isolation
`assertSame / assertEquals`	Vérifier la valeur retournée
`expectException`	Vérifier qu'une exception est levée
`@dataProvider`	Exécuter 1 test avec N jeux de données
`createStub()`	Simuler des valeurs — pas de vérification d'appels
`createMock()`	Simuler + vérifier les appels avec expects()

Ne pas vouloir 100% de couverture — cela n'empêchera pas certains bugs et est très chronophage. Privilégier ce qui est sensible.

Étape 1 — Installer PHPUnit via Composer

CESI ÉCOLE D'INGÉNIEURS

1	**Ouvrir WSL**	Lancer Ubuntu depuis le menu Démarrer ou taper : wsl
2	**Se placer dans le projet**	cd /var/www/html/my_todoo
3	**composer.json est déjà prêt**	La section require-dev avec phpunit/phpunit est en place
4	**Lancer l'installation**	composer install
5	**Vérifier l'installation**	vendor/bin/phpunit --version

WSL — composer install

```
$ cd /var/www/html/my_todoo
$ composer install
Installing dependencies from lock file
  - Installing phpunit/phpunit (11.4.0)
  - Installing sebastian/comparator (6.0.0)
Generating autoload files
$ vendor/bin/phpunit --version
PHPUnit 11.5.55 by Sebastian Bergmann and contributors.
```

composer.json — require-dev

```
"require-dev": {
    "phpunit/phpunit": "^11.4"
}
```

Étape 2 — Fichier phpunit.xml

CESI ÉCOLE D'INGÉNIEURS

phpunit.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:noNamespaceSchemaLocation="vendor/phpunit/phpunit/phpunit.xsd"
         bootstrap="vendor/autoload.php"
         colors="true">
    <testsuites>
        <testsuite name="MyTodoo Test Suite">
            <directory>tests</directory>
        </testsuite>
    </testsuites>
</phpunit>
```

Créer le fichier phpunit.xml

nano /var/www/html/my_todoo/phpunit.xml

Le fichier phpunit.xml est un **fichier de configuration de projet**

Étape 2 — Fichier phpunit.xml

CESI ÉCOLE D'INGÉNIEURS

phpunit.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:noNamespaceSchemaLocation="vendor/phpunit/phpunit/phpunit.xsd"
      bootstrap="vendor/autoload.php"
      colors="true">
   <testsuites>
      <testsuite name="MyTodoo Test Suite">
         <directory>tests</directory>
      </testsuite>
   </testsuites>
</phpunit>
```

Créer le fichier phpunit.xml

nano /var/www/html/my_todoo/phpunit.xml

Le fichier phpunit.xml est un **fichier de configuration de projet**

```
/var/www/html/my_todoo$ ls
composer.json  composer.lock  index.php  phpunit.xml  src  static  templates  tests  vendor
```

Le contenu du fichier TaskModelTest.php se trouve dans le guide du WS-MVC

CESI
ÉCOLE D'INGÉNIEURS

Merci !